Autonomous and Intelligent Systems
Internship report

Josè Dalla Torre
143216@spes.uniud.it

April 2026

Contents

1 Intro **3**

2 Setup **3**

3 Proposed solution **3**
- 3.1 Folder structure . . . 3
- 3.2 Sender . . . 4
- 3.3 Receiver . . . 4
- 3.4 Utils . . . 4

4 Analysis **4**
- 4.1 Data Cleaning . . . 5
- 4.2 Message Reliability . . . 5
- 4.3 Latency Analysis . . . 6
- 4.4 Latency per Message . . . 7
- 4.5 Latency Distribution . . . 8
- 4.6 Tail Latency (99th Percentile) . . . 9
- 4.7 Latency Heatmap . . . 9
- 4.8 RSSI Analysis . . . 9

5 Conclusion **10**

1 Intro

This report presents an evaluation of a LoRa-based communication system developed using Raspberry Pi devices. The objective of this work is to analyze the performance of the LoRa protocol under different environmental conditions and configurations, focusing on key metrics such as message reliability, latency, and signal strength.
The system consists of a sender and a receiver communicating over LoRa, with experiments conducted in both indoor and outdoor scenarios. Various parameters, including spreading factors and environmental obstacles, were tested to understand their impact on communication performance.
The report is structured as follows: first, the system setup and implementation details are described; then, the proposed solution is presented, including the architecture and components. Finally, an in-depth analysis of the collected data is provided, highlighting the strengths and limitations of the system.

2 Setup

This section describes how to correctly set up the operating system on the Raspberry Pi, configure the repository, and install the required libraries for the proposed solution.
After installing the Raspberry Pi OS following official guide, it is necessary to enable SPI (Serial Peripheral Interface). This can be done by running:

```
sudo raspi-config
```

Then navigate to "Interface" then "SPI" and enable it.
Next, install the WiringPi library, which allows high-performance access to GPIO:

```
wget https://github.com/WiringPi/WiringPi/releases/download/3.16/wiringpi_3.16_armhf.deb
sudo dpkg -i wiringpi_3.16_armhf.deb
```

Two files inside the LoRa-RaspberryPi folder must be modified:

- Makefile: ensure #PYTHONVER=python3.11 matches the system's Python version
- lora.c: add the following line at the top for compatibility with newer Python versions: #define PY_SSIZE_T_CLEAN

To check the installed Python version:

```
python3 --version
```

These modifications have already been applied in the updated repository (see the src folder).
Finally, compile the project using:

```
make all
```

3 Proposed solution

This section presents the proposed solution. It begins with an overview of the folder structure, followed by a description of the sender and receiver components. Finally, the data analysis process is discussed.

3.1 Folder structure

Inside the repository there are three folder:

- report: this folder can be ignored since it has been used to store the files needed for this report.
- src: inside this folder there's the code for the receiver and the sender, then the libraries used by both. There's also a utils folder with helper functions for logging and converting digits.
- analysis: inside this folder there's a notebook with the code for the analysis that will be explained later.

3.2 Sender

Inside the sender folder, there are two files: hf-sender.py: initially used to test the maximum message throughput. However, it was not used in the final experiments due to reliability issues.
During testing, it was observed that messages were often corrupted or not correctly received.
After reviewing relevant documentation, the implementation was revised in a new file, sender.py. In this version, message transmission is delayed based on airtime.
Airtime is computed using the formula defined in Semtech's LoRa Modem Designer's Guide (AN1200.13). Additionally, an online tool was used to visualize how airtime is affected by parameters such as payload size and data rate.
To start sending messages use the following command inside the "src" folder:

```
python -m sender.sender
```

3.3 Receiver

Insider the receiver folder there are two files: receiver.py: used to verify message reception and logging.
Initial tests showed that iterating over a fixed number of messages was ineffective due to synchronization issues between sender and receiver.
Therefore, a time-based approach was adopted: messages are sent and received over a fixed duration (2 minutes).
To start the receiver use the following command inside the "src" folder:

```
python -m receiver.hf-receiver
```

3.4 Utils

Inside the utils folder there are two files:

- conversion.py: contains two helper function that convert a positive number to its digit representation in base b.
- logging.py: contains one helper function that create a logger and setup the folders to save the data from the communicator.

The first file has been created to solve the following problem: since the LoRaRaspberryPi library only transmit bytes of data it's necessary to convert the counter to a list of digits in base 255.
It's has been used a list of digits to be able to send a counter greater than 255.

4 Analysis

In this section will be presented the data analysis based on the tests made outside the SMACT 3 laboratory.
In total 7 test has been done:

- first test: communication in the parking area of the laboratory, ∼100 meters. LoRa spreading factor set to 7.
- second test: same as the first test but with a spreading factor set to 12.
- third test: from the water storage outside the canteen to the trees, ∼100 meters, LoRa spreading factor set to 12.
- fourth test: same as the third test but with more obastacles (trees) between the RaspberryPi.
- fifth test: from the tree next to the canteen to "Canale Ledra". LoRa spreading factor set to 12.

- sixth test: communication from Autonomous System laboratory to Cybersecurity laboratory. LoRa spreading factor set to 10.
- seventh test: communication inside the Autonomous System laboratory. LoRa spreading factor set to 10.

4.1 Data Cleaning

The analysis starts with the installation of the required packages and the definition of the function already presented before. The collected data was preprocessed to remove inconsistencies.
This step revealed:

- 5 errors in the sender logs.
- 1 invalid message in the receiver logs.

Given the small proportion of errors relative to the total number of transmitted messages, their impact on the overall results is minimal. Therefore, they are not expected to significantly affect the validity of the analysis.

4.2 Message Reliability

The first part of the analysis evaluates the reliability of the communication by measuring the proportion of successfully received messages.
The results are summarized in 1, which shows the distribution of message outcomes for each test.

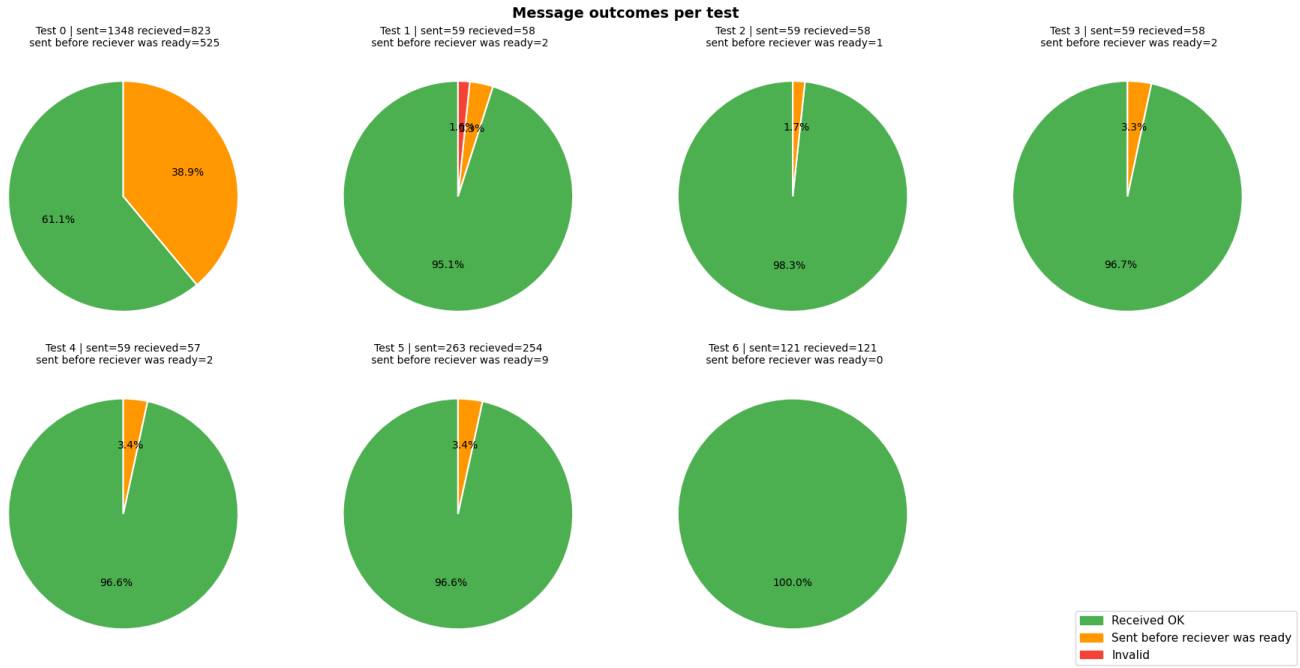

Figure 1: Pie charts of the message outcomes per test

Figure 2 shows the distribution of the Signal to Noise Ratio. Value equals to zero means no errors otherwise an

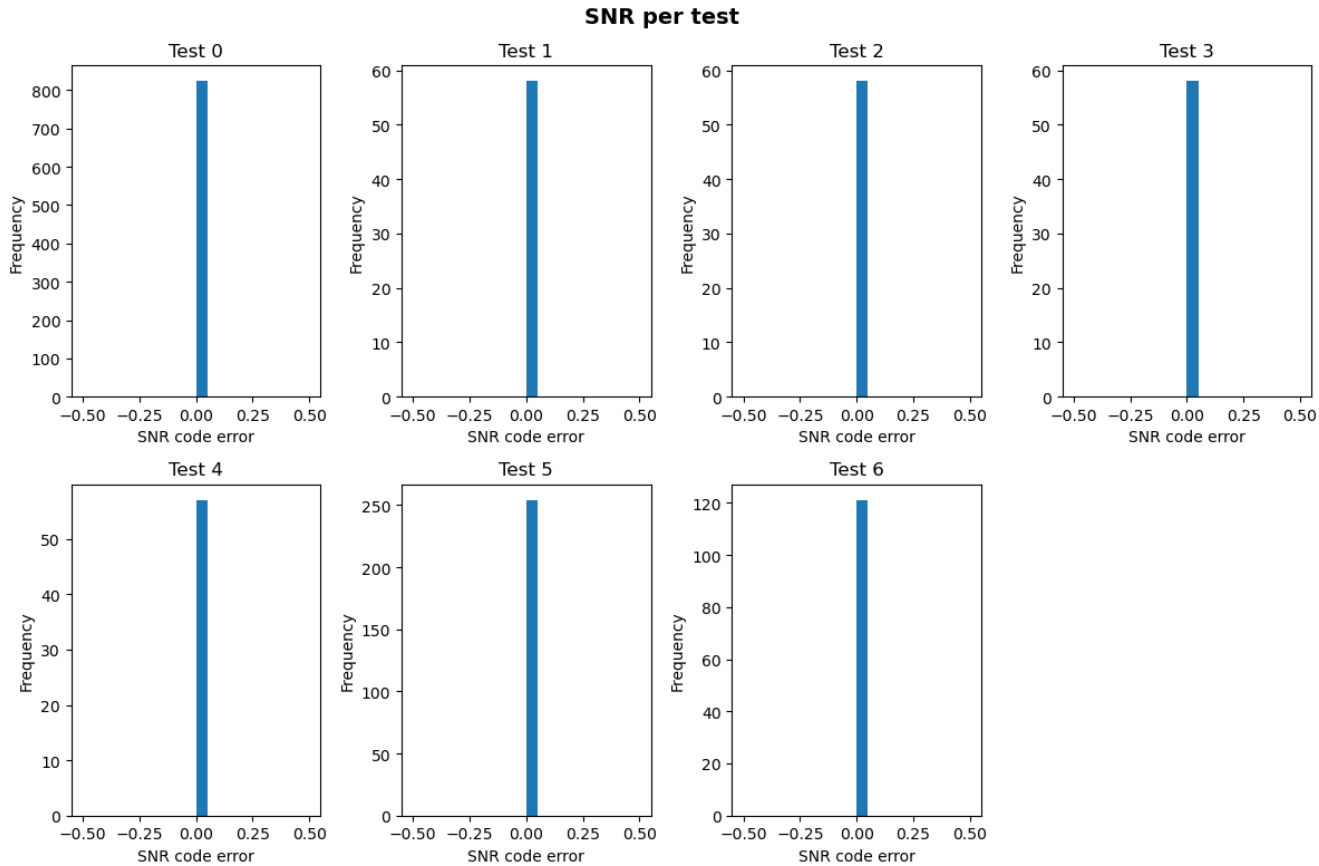

Figure 2: Histogram of the messages SNR per test

error occured.
Since the sender and receiver were manually synchronized, some messages were transmitted before the receiver was ready. These messages are not classified as transmission errors but are instead considered a separate category.
The results indicate that:

- Communication reliability is generally high across all tests.
- Only one invalid message was observed (Test 1).
- Sender-side logging errors occurred sporadically but did not affect reception significantly.

This suggests that the LoRa communication setup is robust under the tested conditions, even in the presence of obstacles and varying spreading factors.

4.3 Latency Analysis

The latency analysis was conducted by matching each received message with its corresponding transmitted message using the message counter. This allowed the computation of end-to-end transmission delay for each successfully

received packet.
Negative latencies and unmatched messages were filtered out to ensure data consistency. In particular, the following metrics were considered:

- Median latency (typical performance).
- Maximum latency (worst-case scenario)
- 99th percentile latency (tail behavior)

These metrics allow the identification of latency spikes and help define acceptable performance bounds.

4.4 Latency per Message

Figure 3 shows the latency for each message across all tests.

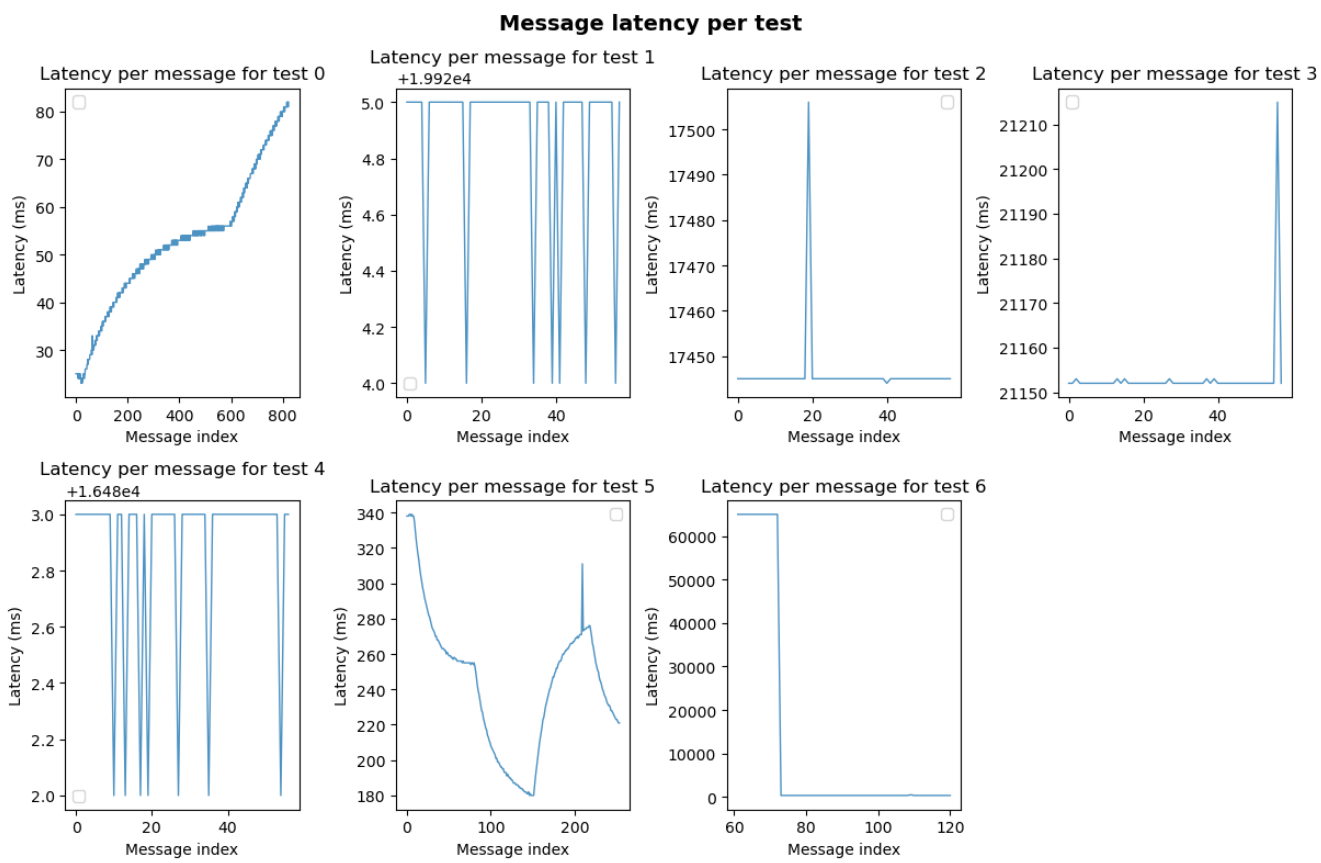

Figure 3: Latency for each message per test

The results highlight that:

- Latency is not constant but varies over time.

- Some tests exhibit occasional spikes, indicating unstable transmission conditions.
- Tests with higher spreading factors tend to show higher latency values.

4.5 Latency Distribution

Figure 4 presents the latency distribution for each test using boxplots.

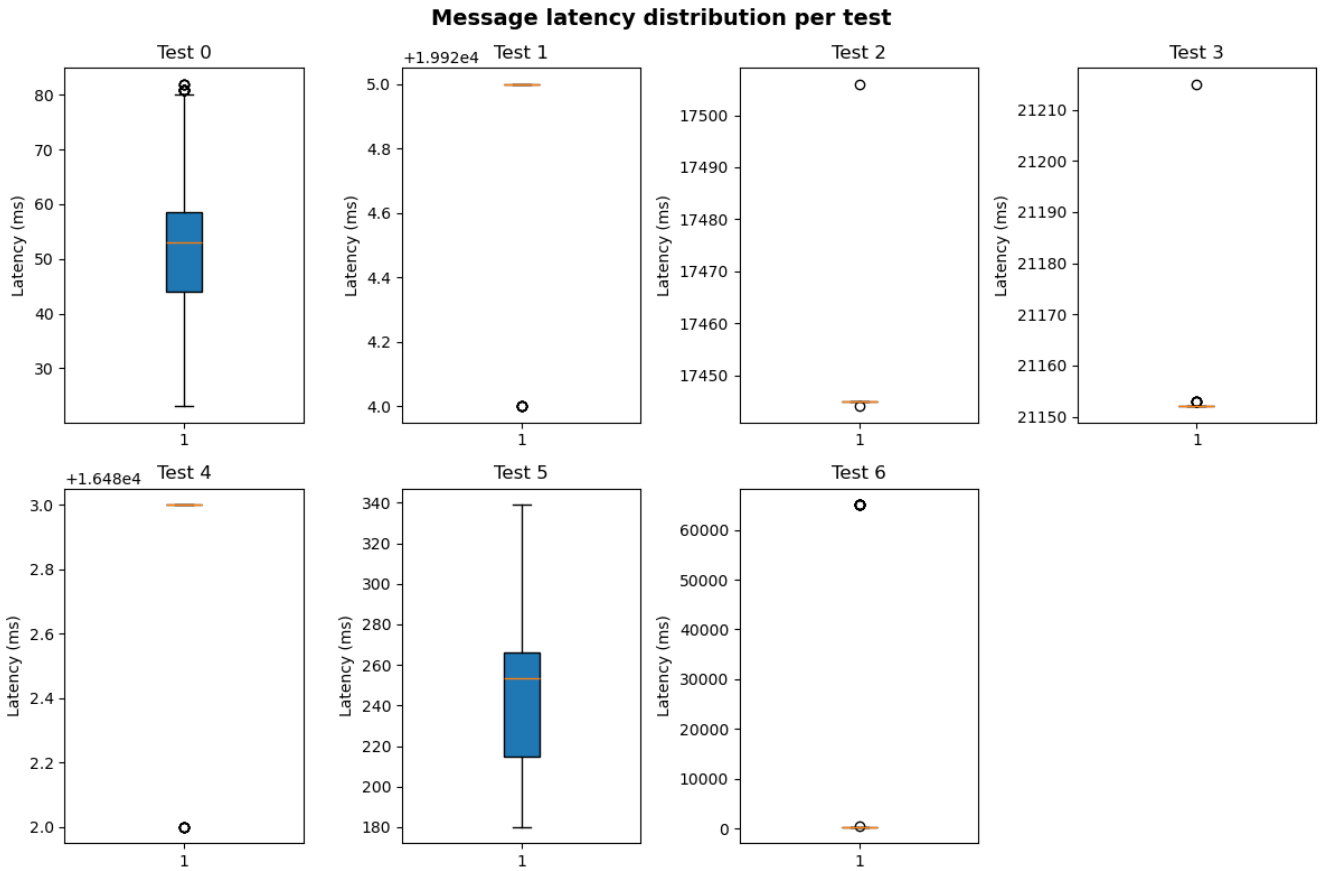

Figure 4: Latency distribution for each test

The analysis shows that:

- The median latency remains relatively stable within each test.
- The interquartile range varies significantly, indicating different levels of variability.
- Some tests contain outliers, corresponding to latency spikes observed in Figure 3.

In particular:

- Tests with higher spreading factors (e.g., SF12) exhibit wider distributions
- Indoor tests show more compact distributions, suggesting more stable communication conditions

4.6 Tail Latency (99th Percentile)

To better understand worst-case performance, the 99th percentile latency was computed for each test.
The results indicate that:

- While average latency remains acceptable,
- Extreme values can be significantly higher, especially in outdoor scenarios.

This suggests that environmental factors (e.g., obstacles, interference) can introduce occasional delays even when overall performance is stable.

4.7 Latency Heatmap

Figure 5 shows a heatmap representation of latency values across all tests. This visualization highlights:

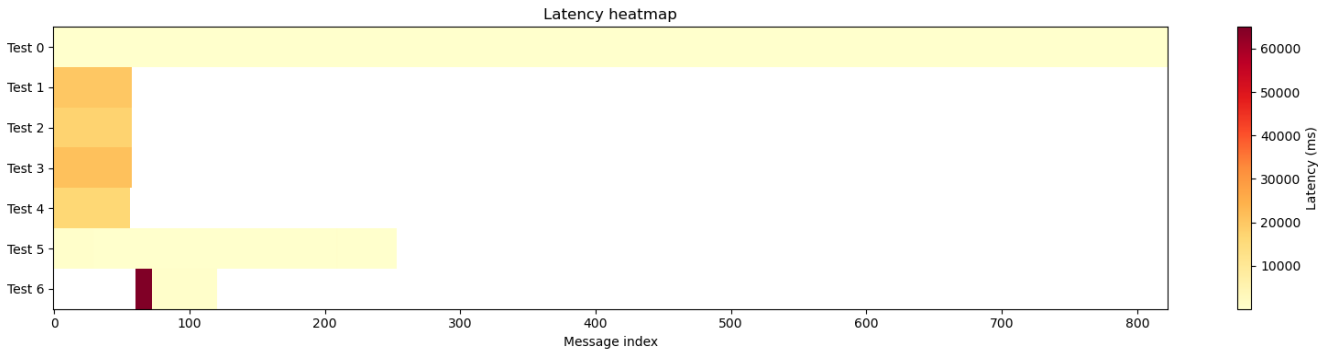

Figure 5: Latency heatmap of the messages latency

- Clusters of higher latency values (hotspots)
- More uniform latency patterns in controlled environments

The heatmap confirms that latency variability is not random but often occurs in bursts, possibly due to temporary interference or transmission delays.

4.8 RSSI Analysis

In addition to latency, the Received Signal Strength Indicator (RSSI) was analyzed for each test.

The PRRSI and RSSI distributions (Figures 6, 7, 8 and 9) show:

- Variability in signal strength across different environments.
- Lower RSSI values in tests with obstacles or longer distances.

The boxplots indicate that:

- Indoor environments tend to produce more stable RSSI values.
- Outdoor tests show wider distributions, reflecting environmental variability.

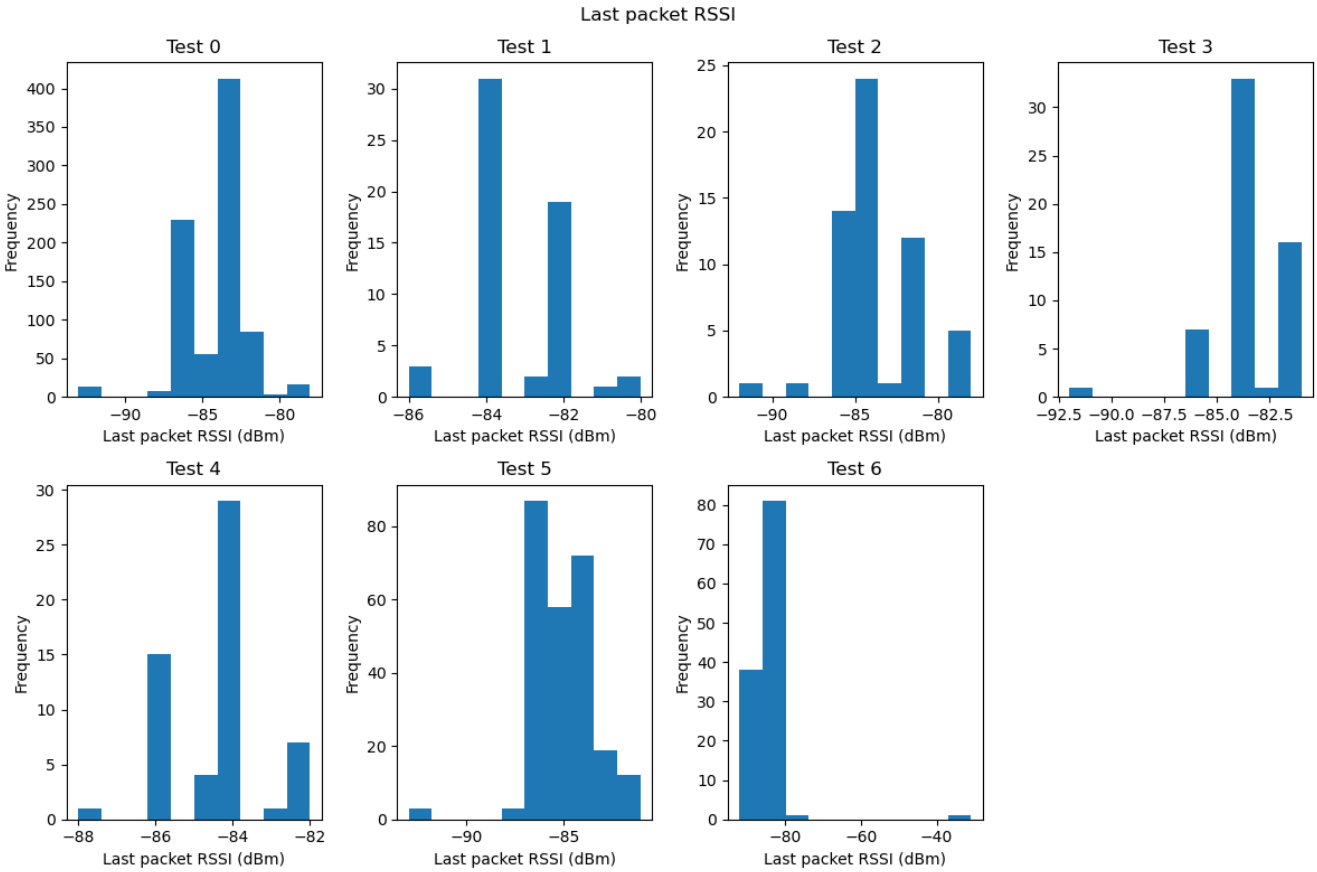

Figure 6: Histogram of the messages PRRSI per test

These results confirm that signal strength directly influences communication reliability and latency.
The analysis shows that:

- Higher spreading factors tend to increase latency due to longer airtime
- Latency variability is more pronounced in outdoor environments with obstacles
- Indoor tests exhibit more stable latency, likely due to controlled conditions

5 Conclusion

This report evaluated the performance of a LoRa-based communication system under different environmental conditions and configurations.
The experimental results demonstrate that the system achieves high reliability, with only a minimal number of transmission errors observed across all tests. This confirms that LoRa is well-suited for long-range, low-power communication even in the presence of obstacles.
Latency analysis revealed a clear trade-off between reliability and performance. Higher spreading factors improve

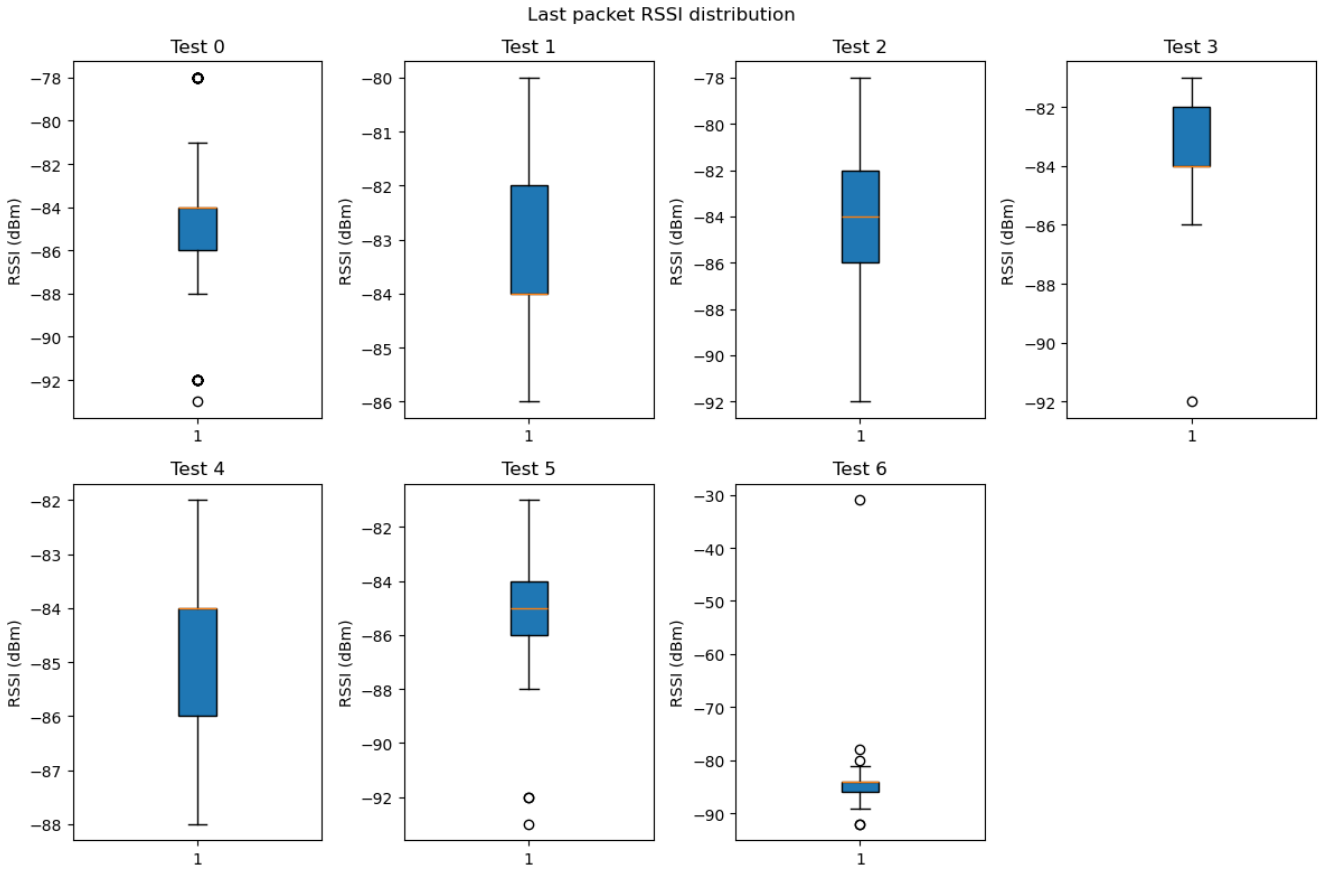

Figure 7: PRRSI distribution for each test

message delivery success but introduce increased latency and variability. This trade-off is a fundamental characteristic of LoRa communication and must be considered when designing real-world applications.
Environmental conditions also play a significant role. Indoor scenarios provide more stable performance, while outdoor environments introduce greater variability due to obstacles and interference.
Overall, the system performs reliably with acceptable latency, making it suitable for low-throughput IoT applications such as monitoring and telemetry.
Future work could focus on:

- Automating synchronization between sender and receiver
- Testing over longer distances
- Evaluating performance under higher network load

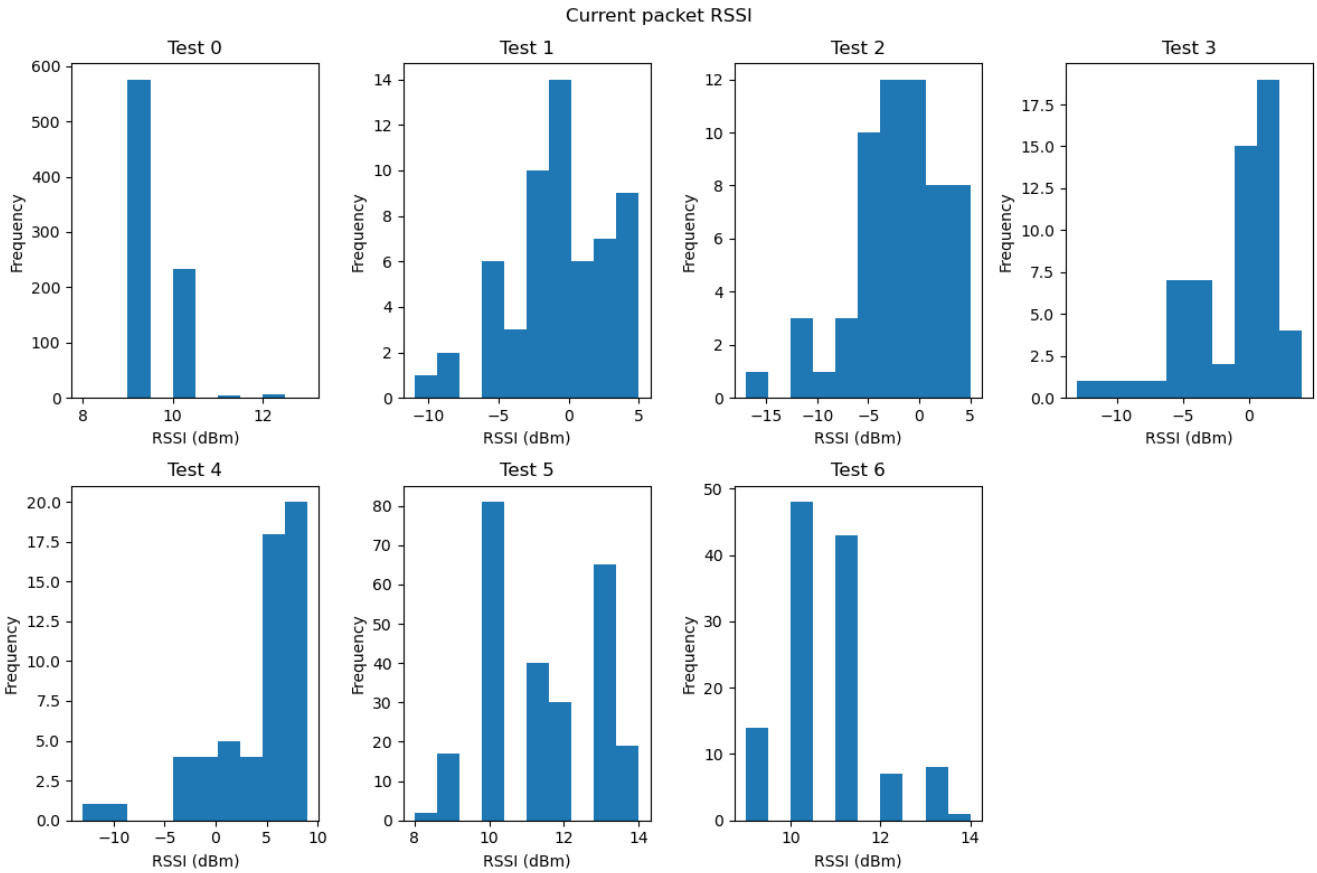

Figure 8: Histogram of the messages RRSI per test

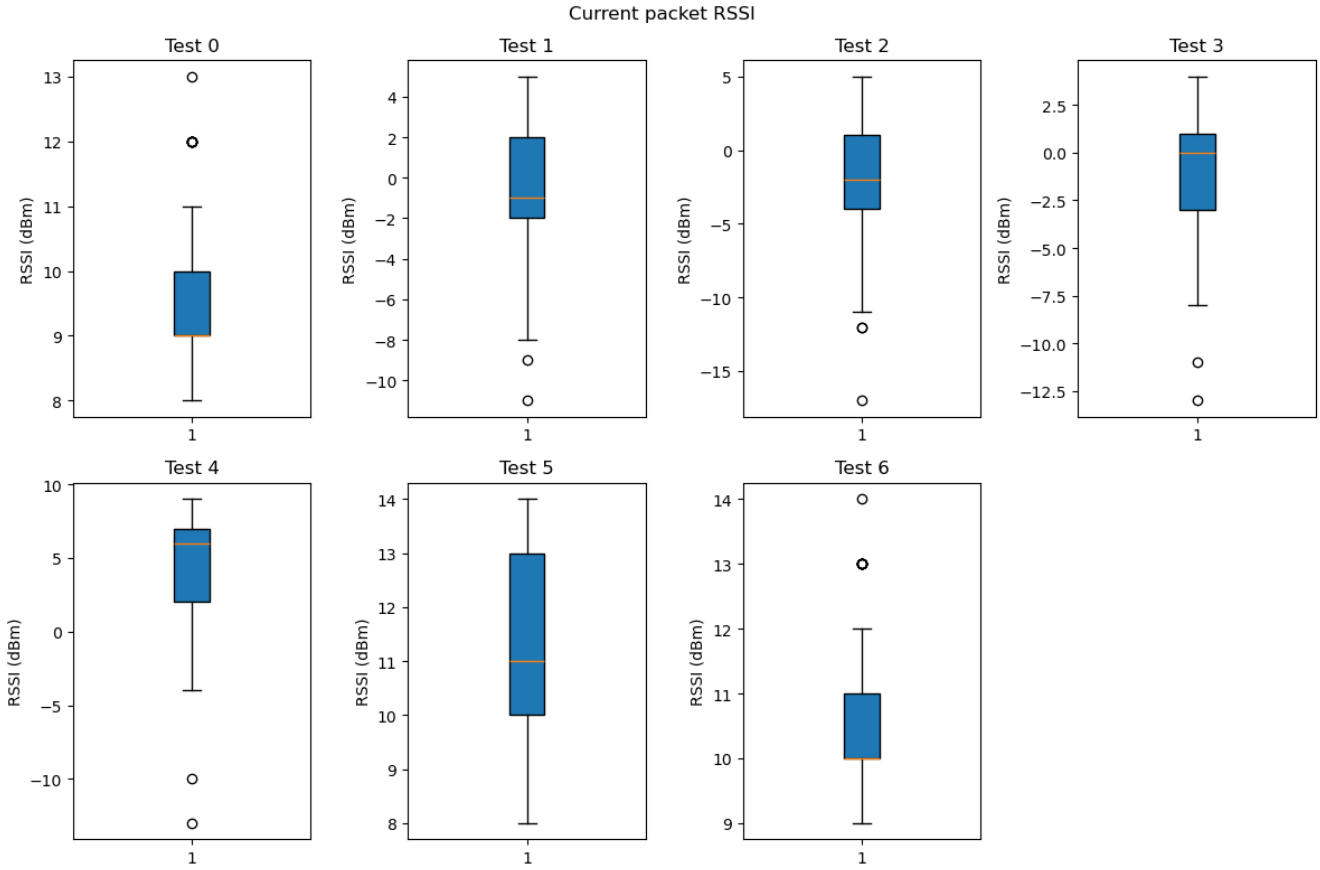

Figure 9: RRSI distribution for each test